

One Ticket, End to End

Tuesday, 10:41. Dispatcher Sunita types one question about KF-2481. Follow it through all eight hops of the control tower — the router, the gate, the connector, the shelves, the belt, the dials, the cameras — and see every Domain 3 objective do its job on one real ticket. Then remove each hop and watch what breaks.

WHAT THIS VOLUME ADDS TO THE EIGHT PARTS

- The building, not the rooms: how the objectives interlock on a single request, in order, with timings
- The actual register lines for the ticket — the JSON the cameras wrote — and the bill in tokens and paise
- The remove-a-hop matrix: what breaks, which objective owns it, and when you would find out
- The demo way vs the dock way, side by side — the sentence every D3 stem is a version of
- Six crossover mini-stems that mix two objectives, the way the real form does

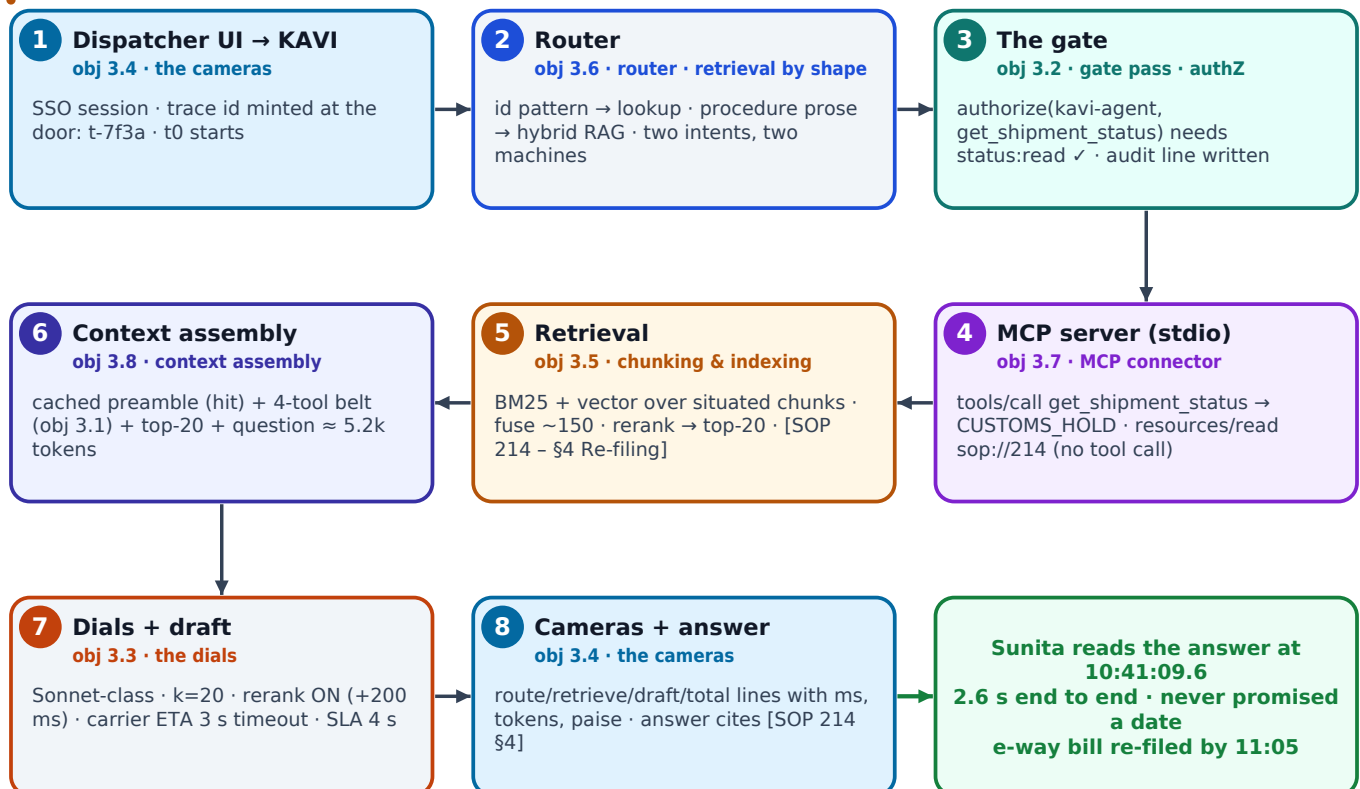
ONE TICKET, END TO END · THE EIGHT OBJECTIVES AS ONE SYSTEM

Tuesday, 10:41 — one question walks through the whole tower

Dispatcher Sunita Kulkarni types into KAVI: 'Where is KF-2481 right now, and what do I do to get it out of customs hold?' Two and a half seconds later she has an answer that cites SOP 214 §4 and does not promise a date. Between those two moments every Domain 3 objective did its job. This volume follows the ticket hop by hop, then removes each hop to show what it was for.

↑ THE BUILDING, NOT THE ROOMS

The eight parts each showed one room of the dock: the gate, the shelves, the cameras. A room makes sense on its own; the **building** only makes sense when a truck drives through it. So here is one truck — ticket t-7f3a — driven through in order, with the clock running and the register writing. The Dock's five laws interlock because the hops interlock: the cameras are what make the belt audit possible; the gate writes to the register; the shelves decide what the dials can buy.



Eight hops, eight objectives, one Tuesday morning. The colour of each box is the colour of its Part.

The eight hops of the tower. Follow the arrows: across, down, back, down, across.

The cast, in one breath

Who / what	Role on this ticket
Sunita Kulkarni	Dispatcher; signed in to the UI by SSO; scopes status:read · draft:write — never refund:write
KAVI	The Kaveri assistant — the agent that calls tools on Sunita's behalf as the service account kavi-agent, carrying her identity
KF-2481	Padma Deshmukh's consignment, in customs hold for a missing e-way bill (the running thread of the Dock)
SOP 214	Customs holds and e-way bill re-filing; nine headed sections; §3 is cancellation, §4 is re-filing
mcp_kaveri_*	The MCP server from Part 1: two tools, the sop://214 resource, the customs_triage prompt
t-7f3a	The trace id minted at the door and stamped on every line this ticket touches

1

The door — identity in, trace id minted · 10:41:07.000

Objective 3.4 · the cameras · Part 7

The UI hands KAVI the question with Sunita's SSO identity attached. The very first thing the handler does is mint a trace id — t-7f3a — and start the clock. Nothing has been answered yet; the **camera** is already following the truck. Every log line from here on carries that id, which is what will let anyone reconstruct this journey out of the 40,000 others that ran today.

What to notice: the identity travels *with* the request. KAVI will call tools as kavi-agent, but the user field says sunita.k — so per-dispatcher rules are possible downstream (Dock Book II, Q7).

2

The router — one question, two shapes · 10:41:07.000 → 07.094

Objective 3.6 · router · retrieval by shape · Part 5

The question has two halves. 'Where is KF-2481 right now' contains an id pattern → the exact world: **structured lookup**. 'What do I do to get it out of customs hold' is procedure prose → **hybrid RAG** over the SOPs. The cheap regex router (Part 5, E6) splits it into two intents in 94 ms and sends each to its own machine. Neither half goes to the wrong one: the status is not guessed from chunks *like* a status, and the procedure is not fetched from a database that does not hold procedures.

Law 3 in action: retrieval follows the shape of the data. Had the whole sentence gone to RAG, Sunita would have read a plausible hold reason composed from SOP text — fluent, cited-looking, wrong.

3

The gate — who you are is not which bays · 10:41:07.094 → 07.100

Objective 3.2 · gate pass · authZ · Part 3

Before the status tool runs, authorize("kavi-agent", "get_shipment_status") checks the scope table: the door needs status:read; kavi-agent holds it. Six milliseconds, and an audit line is written *before* the tool body — it would have been written for a denial too. The order is authorize → audit_log → tool, always.

The ghost that is not here: eighteen months ago carrier-portal-v1 held a wildcard. It was narrowed on Saturday. Tonight at 03:00 that portal will try process_refund and the same gate will write DENY — and the denials panel will spike while latency stays perfectly normal (Part 7, Q22).

4

The connector — a tool call and a resource read · 10:41:07.100 → 07.270

Objective 3.7 · MCP connector · Part 1

KAVI reaches the MCP server over stdio. `tools/call mcp__kaveri__get_shipment_status` with `{consignment_id: "KF-2481"}` returns `CUSTOMS_HOLD`, missing: e-way bill in 140 ms. Then, because the procedure half needs SOP 214 and reading a document should not cost a tool call, the client does `resources/read sop://214` — 30 ms, no action, no side effects. The `customs_triage` prompt, fetched once at session start, is why the draft will end with 'never promise a date'.

Tools act, resources inform, prompts template — all three primitives on one ticket. And it is **one** server: the control tower, Arjun's IDE and the dispatcher UI all dock with the same connector, so when Arjun adds a tool next month, none of them change.

5

The shelves — the right box, whole, labelled from the inside · 10:41:07.270 → 07.790

Objective 3.5 · chunking & indexing · Part 4

The procedure intent is embedded and also keyword-matched. BM25 and the vector index each return 150 candidates from the 3,600 situated chunks; the scores are fused; the reranker (+198 ms) re-sorts; the top 20 go forward. The top box is [SOP 214 v7 – §4 Re-filing] — whole, because the corpus was cut on headings, and self-labelled, because the title and version were written into the chunk text at ingest. §3 (cancellation) is in the shortlist at rank 4, clearly labelled, so the model cannot mistake it for §4.

The incident that did not happen: with fixed 500-token chunks this exact query once returned a box that started mid-§3 and ended before §4's steps, and Padma got a cancelled bill. Retrieval quality was decided six weeks ago at shelving time; today's 520 ms only collects the dividend.

6

The belt and the window — what rides, what was fetched · 10:41:07.790 → 07.810

Objective 3.8 · context assembly · Part 8

Context assembly, 20 ms. The 8,000-token SOP preamble is a **cache hit** (read at ~10% price). The drafting route's belt is **four** tools, not nine — Part 2's audit trimmed it and the logs have not asked for more. The top-20 chunks ride in; so do the two results from hop 4; then the question. About 5,800 tokens, of which the model pays full price for roughly 5,000.

What the model read for this ticket — ≈ 5,800 token-equivalents (progressive, with the cached preamble)



thin slices, left to right after the gold: status + ETA results (220) · the question (140)

The monolith for the same ticket: 9 schemas (2,340) + all 400 SOPs (360,000) + question = 362,480 tokens — 62× more, and no

The window for ticket t-7f3a. The gold slice is the only large one, and it is the one that answers the question.

Law 5 and Law 1 together: the map first (a lean belt, a cached preamble, retrieved chunks), the territory on demand. The monolith would have spent 362k tokens on insurance for a question that needs one tool and one section.

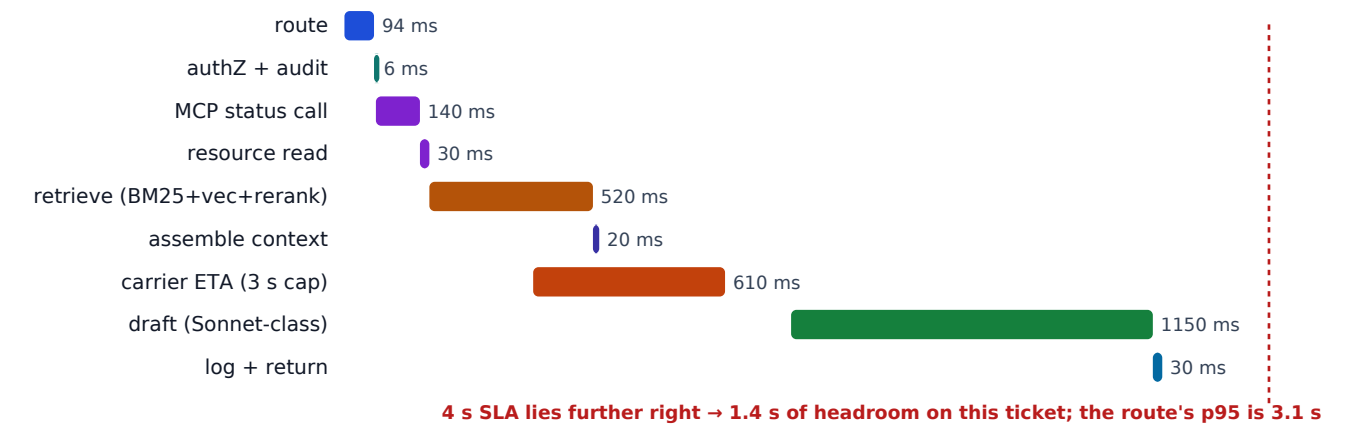
7

The dials — bought, not wished · 10:41:07.810 → 09.570

Objective 3.3 · the dials · Part 6

The drafting route's dial sheet: Sonnet-class model (an error here costs ~₹4, not ₹1,800), $k=20$ (measured best), reranker ON (the 4 s SLA has room for 200 ms), carrier ETA call with a **3 s timeout, one retry, then the cached ETA labelled stale**. The BlueDart ETA call ran concurrently with the tail of retrieval and came back in 610 ms — today. On a cold-API Tuesday it would have been cut at 3 s and the answer would say 'ETA as of 10:15', not hang for 40 s.

Ticket t-7f3a · 10:41:07.0 → 10:41:09.6 · every stage timed alone



The carrier ETA call ran concurrently with retrieval's tail; without the 3 s cap, a cold BlueDart API would have owned the whole SLA.
The swimlane for t-7f3a. Total 2.6 s against a 4 s SLA; the route's p95 is 3.1 s.

The justification, in the exam's shape: 'drafts err at ~₹4 and are 55% of volume → mid-tier model; SOP answers need the correct section ≥ 97% → rerank ON at +200 ms; 4 s SLA holds at p95 3.1 s; blended cost ₹4.90 ≤ ₹6.'

8

The cameras — the register writes, the gauges read · 10:41:09.570 → 09.600

Objective 3.4 · the cameras · Part 7

Every stage wrote one JSON line with the trace id, its milliseconds, its tokens and its cost in paise. The answer goes back to Sunita with a citation to SOP 214 v7 §4 and the flag promised_date: false. Tonight's digest will fold this ticket into the route's p95; tomorrow's bloat audit will count its four tool schemas; the denials panel will see nothing from it. Here is what the register holds for t-7f3a:

register · trace=t-7f3a (nine lines)	
1	{ "trace": "t-7f3a", "stage": "route", "ms": 94, "intents": ["structured_lookup", "hybrid_rag"], "caller": "kavi-agent", "user": "sunita.k" }
2	{ "trace": "t-7f3a", "stage": "authz", "tool": "get_shipment_status", "caller": "kavi-agent", "needed": "status:read", "decision": "ALLOW", "ms": 6 }
3	{ "trace": "t-7f3a", "stage": "tool", "tool": "mcp_kaveri_get_shipment_status", "args": { "consignment_id": "KF-2481" }, "ms": 140, "result": "CUSTOMS_HOLD" }
4	{ "trace": "t-7f3a", "stage": "resource", "uri": "sop://214", "ms": 30, "tokens": 0, "note": "read into context, no tool call" }
5	{ "trace": "t-7f3a", "stage": "retrieve", "ms": 520, "bm25": 150, "vector": 150, "fused": 150, "rerank_ms": 198, "k": 20, "top1": "[SOP 214 v7 - S4 Re-filing]" }
6	{ "trace": "t-7f3a", "stage": "assemble", "ms": 20, "tokens": 5800, "cache_hit": true, "cache_read_tokens": 8000, "belt": 4 }
7	{ "trace": "t-7f3a", "stage": "carrier_eta", "carrier": "BlueDart", "ms": 610, "timeout_ms": 3000, "retrievals": 0, "degraded": false }
8	{ "trace": "t-7f3a", "stage": "draft", "model": "sonnet-class", "ms": 1150, "tokens_in": 5800, "tokens_out": 310, "cost_pc": 41 }
9	{ "trace": "t-7f3a", "stage": "total", "ms": 2600, "cost_pc": 47, "cites": ["SOP 214 v7 S4"], "promised_date": false }

Stage	ms	tokens / equiv.	paise	Which objective priced it
route	94	—	0	3.6 · a regex, not a model
authz + audit	6	—	0	3.2 · set membership in code
MCP status call + resource read	170	0	0	3.7 · a resource costs no tool call
retrieve (BM25 + vector + rerank)	520	3,600 out	3	3.5 / 3.6 · +198 ms for the reranker

Stage	ms	tokens / equiv.	paise	Which objective priced it
assemble (cache hit on 8,000)	20	≈ 5,800 in (≈ 5,000 billed)	—	3.8 / 3.1 · cached preamble, 4-tool belt
carrier ETA	610	—	0	3.3 · 3 s cap, ran concurrently
draft (Sonnet-class)	1,150	5,800 in / 310 out	41	3.3 · mid-tier where errors are cheap
total	2,600		47	3.4 · promised at p95, reported at p95

Forty-seven paise for a correct, cited, dated answer. The Dock's belt-tax arithmetic (Part 2) says the nine-tool belt would have added ~1,500 tokens here; multiplied by today's 40,000 tickets that is the 60-million-token line that ended a meeting.

Now remove one hop at a time

The fastest way to understand what a hop is *for* is to take it away and ask two questions: what breaks, and — the harder one — **when would you find out?** Observability gaps do not error; they wait.

REMOVE ONE HOP	OBJ	WHAT BREAKS — AND WHEN YOU WOULD FIND OUT
Drop the trace id (hop 1)	3.4	digest() still prints; 'where did t-7f3a lose time' is unanswerable; the cold-API tail becomes invisible
Skip the scope check (hop 3)	3.2	the same call succeeds today — and carrier-portal-v1's wildcard reads every customer's holds at 03:00
Route both intents to RAG (hop 2)	3.6	the status question returns chunks LIKE a status; Sunita reads a plausible, wrong hold reason
Fixed 500-token chunks (hop 5)	3.5	§3 cancellation and §4 re-filing share a box; the answer cancels Padma's bill (the original incident)
All 9 tools on the belt (hop 6)	3.1	+1,500 tokens on this and 39,999 other calls today; 7% chance the model dials lookup_order
Stuff the corpus instead (hop 6)	3.8	362k tokens per call; the window is full before the question is read
No carrier timeout (hop 7)	3.3	a cold BlueDart API holds the ticket 40 s; SLA gone; the dispatcher refreshes and doubles the load
Log free text (hop 8)	3.4	no p95 by stage; the belt tax, the denial spike and the tail are all invisible

Eight removals, eight objectives. Notice how often the answer to 'when would you find out' is 'much later'.

⚠ TRAP · 'IT WORKED IN THE DEMO'

Every removal above is something the Friday demo did not have, and the demo worked anyway: one user, ten documents, one key taped over every door, a developer watching the console. Every Domain 3 stem is some version of the distance between that Friday and this Tuesday. When an option's defence is that it worked at small scale, look for what breaks at 40,000 a day.

THE DEMO WAY (Friday)

- one user, ten SOPs pasted into the prompt
- one API key taped over every door (scope *)
- every question goes to the vector index
- fixed 500-token chunks, top-5
- no timeouts; the developer watches the console
- print('done') is the observability

THE DOCK WAY (Tuesday)

- 40 dispatchers, 400 SOPs, 40,000 calls a day
- per-caller scopes checked in code; denials logged
- router: lookup / aggregate / hybrid by shape
- heading chunks, situated, hybrid + rerank, top-20
- 3 s timeouts, one retry, honest degradation
- trace id, JSON lines, p95 by stage, few dear alerts

Same ticket, two systems. The right column is what the eight parts built.

How the five laws hold each other up

This law...	...is only possible because...	...and it feeds...
1 · Every tool is a door (3.1 + 3.2)	the cameras logged which tools were called (3.4)	the belt that rides in the window (3.8)
2 · Chunk along meaning (3.5)	the corpus was big enough to need RAG (the $\leq 200k$ pre-question)	what the dials can buy at query time (3.3, 3.6)
3 · Retrieval follows shape (3.6)	the router in front is cheap and known-category (D1)	which machine, and therefore which bill, each intent pays
4 · You cannot fix what you cannot see (3.4)	a trace id was minted at the door and every stage timed alone	every number in Parts 2, 3, 6 and 8
5 · Map first, territory on demand (3.8)	the connector can list before it calls (3.7) and retrieval replaces stuffing (3.5)	a window with room to think

Read the middle column twice. It is the reason the exam mixes objectives inside one stem: in a real system no objective stands alone, and the correct option is usually the one that respects the hop *before* the one the stem names.

Six crossover mini-stems — two objectives each, answered

Distractor families (why a wrong option is wrong): **OE** over-engineered · **VG** vocabulary gap · **TR** trade-off reversed · **CM** constraint missed · **HL** human loop skipped · **SF** silent failure · **SD** sounds-diligent · **OP** over-permissive.

3.2 + 3.4

The denials panel spikes for kavi-agent on process_refund from the drafting route; latency is normal; no refund went out.

Answer: Not an attack and not noise: the gate is working and the belt on that route now offers a door it should not. Remove the tool from the route's belt; leave the scope denied; keep the panel.

The gate feeds the register; the register feeds the panel; the panel names a belt problem. Three objectives, one line of JSON.

3.5 + 3.3

A stem fixes a 4 s SLA at p95 3.85 s and offers 'add contextual retrieval at ingest to cut failures 49%'.

Answer: Accept — situating runs at ingest, weekly, and adds nothing to the query path; the latency currency is untouched.

Reranking would breach (+200 ms); situating would not. Know where each dial's cost lands before you refuse it.

3.6 + 3.8

A 12k-row tariff table is proposed for 'the cached prefix so the model can look up HSN codes'.

Answer: Neither cache nor RAG: keyed rows want a structured lookup tool. Stuffing a table into a prefix turns exact answers into the model's recall.

Law 3 outranks Law 5: the data's shape picks the machine before the context strategy is even a question.

3.7 + 3.2

Three teams will reach the SOP server over Streamable HTTP; the stem asks for the mechanism AND the auth.

Answer: One MCP server (many clients, changing list) with OAuth-issued, per-team tokens scoped sop:read and rotated — the transport decides the authN, the table decides the authZ.

3.7 picks the connector; 3.2 puts the gate on it. A stem that names a remote transport is asking both.

3.1 + 3.8

30 tools across aggregated servers; any request uses at most 3; the agent tier's wrong-tool rate is rising.

Answer: A per-route default belt (3.1) plus a one-line catalogue for the rest, fetched on choice (3.8): 7.8k standing tokens become ~1.5k and the lookalikes leave the belt.

Trim what the logs never use; catalogue what is used but rarely; keep what is always used in the prefix.

3.4 + 3.3

Leadership declares the retrieve stage healthy at mean 310 ms; the SLA is 3 s end to end; p95 is 2.9 s.

Answer: The SLA is a promise about the tail: 5% of dispatchers already live at 2.9 s in retrieval alone. Fix the cold-carrier tail (timeout + cached ETA) — do not touch the mean.

3.4 says which number to read; 3.3 says which dial to turn once you have read it.

Day D exercise — redraw the building from memory

Cover the first page. On a blank sheet draw the eight hops in order with their objective numbers, then write one line per hop: what it does, what it costs (ms and tokens), and what breaks without it. Check against the map and the remove-a-hop matrix. Any hop you could not place is the part to re-read tonight.

THE EIGHT HOPS IN ONE BREATH

1. Door: identity in, trace id minted (3.4).
2. Router: id → lookup; procedure prose → hybrid RAG (3.6).
3. Gate: authorize → audit_log → tool, denials logged (3.2).
4. Connector: tools/call for status; resources/read for SOP 214; prompt for the procedure (3.7).
5. Shelves: heading chunks, situated, BM25 + vector, fuse ~150, rerank, top-20 (3.5).
6. Belt + window: lean belt, cached preamble, retrieved chunks, question (3.1 + 3.8).
7. Dials: mid-tier where errors are cheap, rerank where the SLA allows, 3 s timeouts (3.3).
8. Cameras: one JSON line per stage with ms, tokens, pause; p95 not means (3.4).

Next sitting: Dock Book II, cold, ninety minutes. Every one of its forty-five stems lives somewhere on this map.